Assignment 02

Linear Regression and Logistic Regression using scikit-learn library

Khiem Nguyen Lecturer in Multiscale Materials khiem.nguyen@glasgow.ac.uk

Table of Content

1 Linear regression using `LinearRegression` 1
1.1 Predict the strength of concrete . . . 1
1.2 Build a polynomial regression model . . . 3
1.3 Polynomial regression model for `concrete` data. . . . 6

2 Logistic regression using `LogisticRegression` 7
2.1 Electrocardiogram (ECG) anomaly detection . . . 8
2.2 Hand-written digit classification . . . 11

Before we discuss the mathematics behind Linear Regression and Logistic Regression, let us examine two examples given in the two above-mentioned files `concrete.csv` and `ecg.csv`. We first write a Python program to make prediction of concrete strength and then another program to make prediction whether a patient is fine or not given their ECG data.

1 Linear regression using `LinearRegression`

We shall study two applications of `LinearRegression`

- Predict the concrete strength using 8 features of technical info of the concrete.
- Build a simple polynomial prediction model.

1.1 Predict the strength of concrete

Load data from `csv` file

First, we need to load data using the library Pandas and then convert the data into `ndarray`.

```
import pandas as pd
import numpy as np
df = pd.read_csv("./data/concrete.csv") # concrete_data is a DataFrame object
data = df.to_numpy()    # now, it is a ndarray object
print(f"First 5 rows of this concrete dataset:\n {df.head()}")
```

```
First 5 rows of this concrete dataset:
    cement   slag  ash  water  superplastic  coarseagg  fineagg  age  strength
0   540.0    0.0  0.0  162.0           2.5     1040.0    676.0   28     79.99
1   540.0    0.0  0.0  162.0           2.5     1055.0    676.0   28     61.89
2   332.5  142.5  0.0  228.0           0.0      932.0    594.0  270     40.27
3   332.5  142.5  0.0  228.0           0.0      932.0    594.0  365     41.05
4   198.6  132.4  0.0  192.0           0.0      978.4    825.5  360     44.30
```

This is a pretty sizable dataset. Both input features $\mathbf{x}^{(i)}$ and output label $y^{(i)}$ are stored in one single variable `data`. The first 8 columns correspond to the input $\mathbf{x}$ and the last column to the output y. We extract `X` and `y` from this variable `data`:

```
X = data[:,:-1]     # all rows, all columns from the first to the second last
y = data[:,-1]      # all rows, last column
```

Train the model

You will learn that we need to test whether our prediction model is good or not. However, all the data we have are only avaliable in the `csv` file. For this reason, machine learning practioners normally split the dataset into two sets: **training set** and **test set**. The training set is used to train the machine learning model, which means seeking the model parameters $\mathbf{w}$ and b explained in Assignment 1. In our project, we don't have to training-test split when we build the library. I just want to explain the point.

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score    # to evaluate how good the model is later

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42)
```

Training the model by using class `LinearRegression` from **scikit-learn** is ridiculously easy as all the power-lifting has been done by the library.

```
model = LinearRegression()  # define a LinearRegression model
model.fit(X_train, y_train) # training happens here -- look for model parameters w, b
```

fit_intercept	True
copy_X	True
tol	1e-06
n_jobs	None
positive	False

Coefficient of determination

Now that our model has been trained. In 1D and 2D linear regression, we can visualize the linear regression model and the scattered data to see how good the model is. However, this approach can lead to deceptive result and cannot be applied to higher dimensional regression models. We need to assess how good our prediction model is by using a numerical measurement which is just a scalar. To do this, we compute the **coefficient of determination**, denoted R^2 and pronounced "R squared", on both the training set and the test set. We normally call this measurement R^2 scaore. The R^2 score is defined by

$$R^2 = 1 - \frac{\sum_{i=1}^{m}(y^{(i)} - \hat{y}(\mathbf{x}^{(i)}))^2}{\sum_{i=1}^{m}(y^{(i)} - \overline{y})^2} = 1 - \frac{S_{\text{residual}}}{S_{\text{total}}}$$

where $\overline{y}$ is the mean value

$$\overline{y} = \frac{1}{m}\sum_{i=1}^{m} y^{(i)}.$$

By denoting

$$S_{\text{residual}} = \sum_{i=1}^{m}(y^{(i)} - \hat{y}(\mathbf{x}^{(i)}))^2, \qquad S_{\text{total}} = \sum_{i=1}^{m}(y^{(i)} - \overline{y})^2,$$

we can rewrite R^2 score as

$$R^2 = 1 - \frac{S_{\text{residual}}}{S_{\text{total}}}$$

As we can see, if our prediction model is very good, the nominator S_{residual} will be pretty small because the predicted value $\hat{y}(\mathbf{x}^{(i)})$ is very close to the actual label $y^{(i)}$, making the entire fraction $\frac{S_{\text{residual}}}{S_{\text{total}}}$ is pretty small and hence the R^2 is close to 1. On the other hand, if our model makes bad predictions, the nominator S_{residual} is rather large, making the entire fraction $\frac{S_{\text{residual}}}{S_{\text{total}}}$ is large too. In extreme cases where the prediction model is unreasonably bad, this fraction can be larger than 1 and thus R^2 score can be negative.

Summary

The R^2 score being close to 1 means the model predicts extremely well, the score being close to 0 means the model predicts extremely bad. Note that R^2 score can be negative – The square term may be misleading you to think that it must be a non-negative number.

```
r2_score_train = r2_score(y_true=y_train, y_pred=model.predict(X_train))
r2_score_test = r2_score(y_true=y_test, y_pred=model.predict(X_test))
print(f"Training set: R2-score: {r2_score_train:.4f}")
print(f"    Test set: R2-score: {r2_score_test:.4f}")
```

```
Training set: R2-score: 0.6105
    Test set: R2-score: 0.6276
```

According to these R^2-score, the model has done a not so good job in predicting the concrete strength given 8 input features. We can significantly improve the prediction model by adding the polynomial features such as

$$x_1^2, x_2^2, \dots, x_8^2, x_1x_2, x_1x_3, \dots x_1x_8, x_2x_3, x_2x_4, \dots, x_2x_8, x_3x_1, \dots$$

Remark

ENG3091 is not a Machine Learning course. We don't aim to obtain a very good prediction model. Instead, we aim to build a reliable codebase and program. For this reason, a model that could achieve good result as this benchmark is fine.

1.2 Build a polynomial regression model

Following the above discussion, we study a bit about polynomial regression models. In this example, we generate a dataset with one single feature by adding noise to data points created on the graph of a cubic function

$$p(x) = x(x-1)(x+1) = x^3 - x \tag{1}$$

You should see that the application and steps laid out here can be generalized to polynomial regression models for inputs of multiple features. We only need to form the polynomial features $x_1^2, x_1x_2, x_1x_3, \dots x_2^2, x_2x_3, x_2x_4, \dots$.

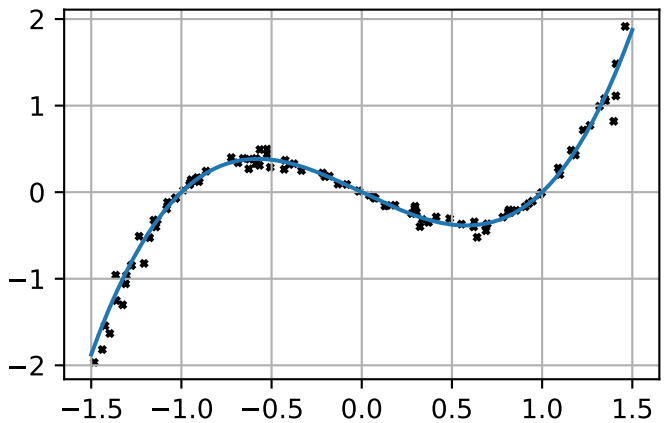
Generate synthesized data

As a starting point, we generate synthesized data with noise based on the cubic polynomial Equation 1.

```
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(42)
n_samples = 100
poly_true = lambda x: x * (x - 1) * (x + 1)
x_data = np.random.uniform(-1.5, 1.5, n_samples)
```

```
y_true = poly_true(x_data)
noise = np.random.normal(0, 0.2, size=n_samples) * y_true
y_data = y_true + noise
xx = np.linspace(-1.5, 1.5, 1000)

plt.figure(figsize=(4, 2.5))
plt.plot(xx, poly_true(xx))
plt.scatter(x_data, y_data, marker='x', c='k', s=6)
plt.grid(True)
```



As we can see, at this point we have the dataset

$$\mathcal{D} = \left\{(x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)})\right\}$$

with $x^{(i)} \in \mathbb{R}$. Apparently, if we try to fit a line to this dataset, *the fitting line has no chance to really fit the data well. One cannot "fit" a straight line to a curve.* To build a model that fits this kind of data, we apparently that the following function will do a pretty good job

$$f_{\mathbf{w},b}(\mathbf{x}) = w_1 x + w_2 x^2 + w_3 x^3 + b = \mathbf{w} \cdot \mathbf{x} + b, \tag{2}$$

where

$$\mathbf{w} = (w_1, w_2, w_3), \quad \mathbf{x} = (x, x^2, x^3).$$

You should see immediately from Equation 2 that "Wait a minute", that's exactly a linear regression model with 3 features, each feature x_j is formed by taking the original feature x to power j. That is, we have $x_1 = x^1, x_2 = x^2, x_3 = x^3$.

Generate polynomial features

It is easy to form polynomial features in NumPy:

```
X_poly_np = np.vstack((x_data, x_data**2, x_data**3)).transpose()
```

Fortunately, **scikit-learn** gives us a tool to do this too. In the following code, the feature $x^0 = 1$ is also added to the matrix `X_poly`.

```
from sklearn.preprocessing import PolynomialFeatures
poly_features = PolynomialFeatures(degree=3)
X_data = x_data.reshape((-1, 1))
X_poly = poly_features.fit_transform(X_data)
print(f"X_poly =\n{X_poly[:4]}") # print the first 4 samples
print(f"X_poly.shape = {X_poly.shape}")
```

```
X_poly =
[[ 1.         -0.37637964  0.14166164 -0.05331856]
 [ 1.          1.35214292  1.82829047  2.47211002]
 [ 1.          0.69598183  0.4843907   0.33712712]
 [ 1.          0.29597545  0.08760147  0.02592788]]
X_poly.shape = (100, 4)
```

Since we don't need the bias feature x^0, let us redo this step without this bias feature

```python
poly_features = PolynomialFeatures(degree=3, include_bias=False)
X_poly = poly_features.fit_transform(X_data)
print(f"X_poly =\n{X_poly[:4]}") # print the first 4 samples
print(f"X_poly.shape = {X_poly.shape}")
```

```
X_poly =
[[-0.37637964  0.14166164 -0.05331856]
 [ 1.35214292  1.82829047  2.47211002]
 [ 0.69598183  0.4843907   0.33712712]
 [ 0.29597545  0.08760147  0.02592788]]
X_poly.shape = (100, 3)
```

We can easily check whether `X_poly_np` created above agress with `X_poly` created by using `PolynomialFeatures`.

```python
# X_poly_np created above and this X_poly should be identical
print(f"X_poly[:4] - X_poly_np[:4] =\n{X_poly[:4] - X_poly_np[:4]}")
print(f"X_poly is close to X_poly_np = {np.allclose(X_poly, X_poly_np)}")
```

```
X_poly[:4] - X_poly_np[:4] =
[[0.0000000e+00 0.0000000e+00 6.9388939e-18]
 [0.0000000e+00 0.0000000e+00 4.4408921e-16]
 [0.0000000e+00 0.0000000e+00 0.0000000e+00]
 [0.0000000e+00 0.0000000e+00 0.0000000e+00]]
X_poly is close to X_poly_np = True
```

Train the model

The synthetical dataset is ready. We can now proceed to build the linear regression as in the last application. Note that we don't split the dataset into the training dataset and the test dataset because we don't want to verify the model against the test data. Instead, we just plot the polynomial regression model.

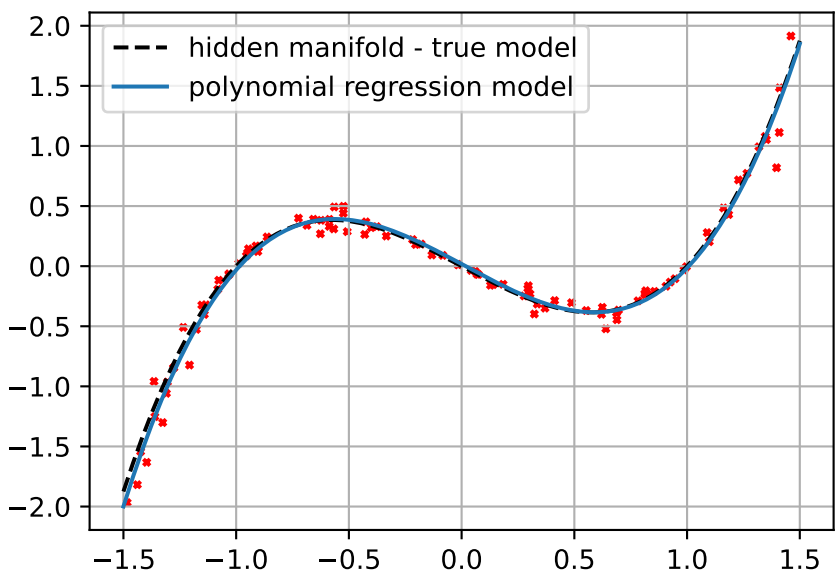
```python
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_poly, y_data)
```

fit_intercept	True
copy_X	True
tol	1e-06
n_jobs	None
positive	False

Plot the polynomial regression model

If you think about this whole process "long enough", you will see that the linear regression model we have just built is nothing but just a function of multiple variable. That is, given a vector $\mathbf{x} = (x_1, x_2, x_3)$ this model gives me the value $\hat{y} = f_{\mathbf{w}}, b(\mathbf{x})$. That's it. Therefore, to justify the prediction power of this model, let us try to visualize it. Hopefully, it should agree well with the original polynomial $p(x) = x(x-1)(x+1)$. However, as just explained, this model is a function of three variables. To visualize it, we must provide the input the inform (x, x^2, x^3). This is exactly what we will achieve in the following code.

```
xx = np.linspace(-1.5, 1.5, 500)
XX = poly_features.fit_transform(xx.reshape(-1, 1))
plt.figure(figsize=(5, 3.5))
plt.plot(xx, poly_true(xx), 'k--', label="hidden manifold - true model")
plt.plot(xx, model.predict(XX), label='polynomial regression model')
plt.scatter(x_data, y_data, marker='x', s=6, c='r')
plt.legend()
plt.grid(True)
```



Here we go. The polynomial regression model closely fits the true (hidden) manifold that is used to generate the synthetic and random data.

1.3 Polynomial regression model for `concrete` data.

Let us make a **quick revisit** to the regression problem of predicting the concrete strength and the dataset given by `/data/concrete.csv`. There won't be a lot of explanation as most of the code is almost identical to what we learned above.

In this example, we shall build second-degree polynomial regression model. Since we have originally 8 input features, the second-degree polynomial regression model will involve 44 features. This number 44 is explained as follows.

- As for first-degree features we have
  - 8 first-degree features: $x_1, x_2, x_3, \dots x_8$
- As for second-degree features we have
  - 8 second-degree features: $x_1^2, x_1x_2, x_1x_3, \dots x_1x_8$
  - 7 second-degree features: $x_2^2, x_2x_3, \dots x_2x_8$
  - Similarly, we shave have $6, 5, 4, 3, 2, 1$ second degree features corresponding $x_3^2, x_3x_4, \dots$ and so on

In total, we have

$$\underbrace{8}_{\text{first-degre}} + \underbrace{8+7+6+...+1}_{\text{second-degree}} = \frac{8*(8+1)}{2} + 8 = 36 + 8 = 44$$

Generate data with polynomial features

```
df = pd.read_csv("./data/concrete.csv")
data = df.to_numpy()
X, y = data[:,:-1], data[:,-1]
print(f"X.shape = {X.shape}")

poly_features = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly_features.fit_transform(X)

print(f"X_poly.shape = {X_poly.shape}")
```

```
X.shape = (1030, 8)
X_poly.shape = (1030, 44)
```

Train the model

```
X_train, X_test, y_train, y_test = train_test_split(X_poly, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)
```

fit_intercept	True
copy_X	True
tol	1e-06
n_jobs	None
positive	False

Assess the model

```
r2_score_train = r2_score(y_true=y_train, y_pred=model.predict(X_train))
r2_score_test = r2_score(y_true=y_test, y_pred=model.predict(X_test))
print(f"Training dataset -- R2-score: {r2_score_train}")
print(f"    Test dataset -- R2-score: {r2_score_test}")
```

```
Training dataset -- R2-score: 0.8130773700411071
    Test dataset -- R2-score: 0.7842939778490666
```

It is clear that the polynomial regression model for predicting concrete strength performs better than a simple linear regression model we trained before. R^2 score this time is better: 0.782 as compared to

2 Logistic regression using `LogisticRegression`

We will study two applications of `LogisticRegression`

- Classify Electrocardiogram dataset into two classes: normal or abnormal
- Classify gray-scale hand-written images into 10 classes: digits $0, 1, \ldots, 9$

2.1 Electrocardiogram (ECG) anomaly detection

In this application, given an electrocardiogram of a patient we try to predict whether the patient is fine or not. This is a binary classification problem in which a data sample is classified as either 0 or 1, "yes" or "no". The following Python code snippet will walk you through how the `LogisticRegression` from **scikit-learn** perform binary classification.

Load the dataset

As before, we need to load the csv file into an `ndarray` to prepare the data for training the logistic regression model. The dataset consists of 4997 samples, each sample has 140 input features and one single output label taking value either 0 or 1.

In our ECG example, the label values are interpreted according to - label equal to 1: **normal** – the patient is fine - label equal to 0: **abnormal** – the patient is **not fine**

```
import pandas as pd
df = pd.read_csv("./data/ecg.csv")  # df is a dataframe
data = df.to_numpy()    # convert dataframe into numpy array
print(f"type(df) = {type(df)}")

X = data[:,:-1] # all the features in the first 140 columns
y = data[:,-1]  # label of data (y vlaue) in the last column
n_samples = X.shape[0]          # number of samples
n_features = X.shape[1]         # number of features
n_classes = len(np.unique(y))   # number of classes in the classification problem

# Number of labels equal to 0
print(f"number of labels = 0: {np.sum(y == 0)}")
print(f"number of labels = 1: {np.sum(y == 1)}")
```

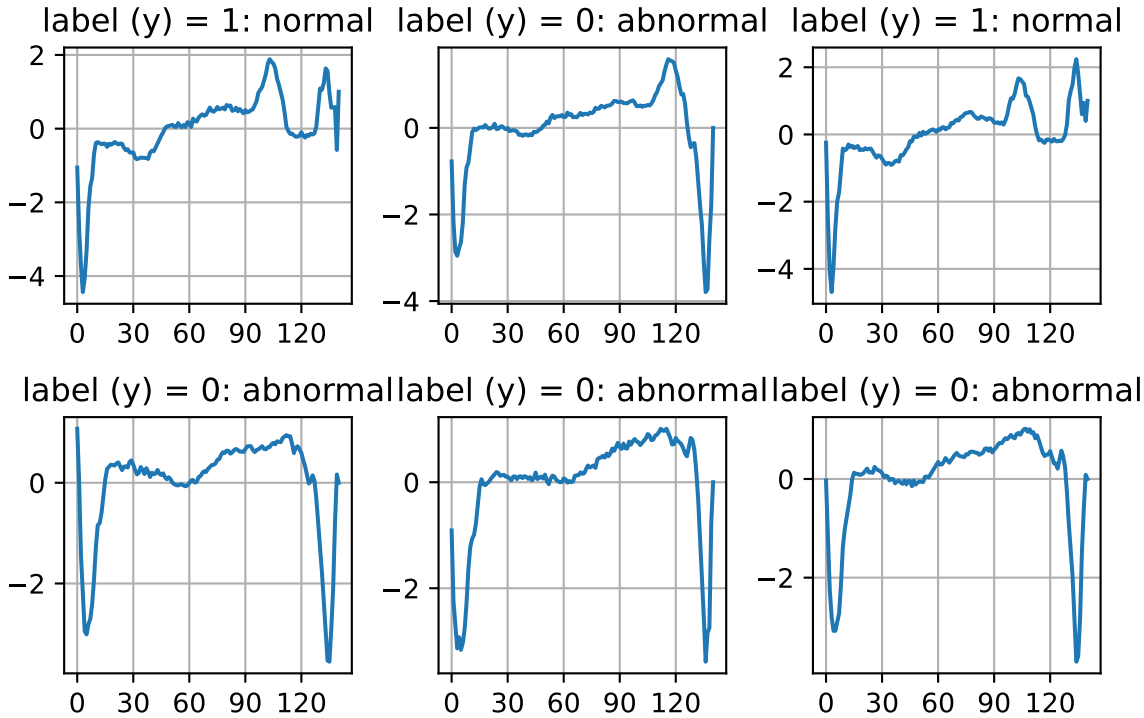
```
type(df) = <class 'pandas.DataFrame'>
number of labels = 0: 2079
number of labels = 1: 2918
```

We now plot some data samples and their corresponding label (either 0 (abnormal) or 1 (normal)).

```
import matplotlib.pyplot as plt
import numpy as np
# Plot some of the data for better understanding
nrows, ncols = 2, 3     # 2 rows, 3 columns for 6 subplots
plt.figure(figsize=(6, 4))
np.random.seed(seed=42)         # just for reproducibility purpose

for i in range(nrows * ncols):
    plt.subplot(nrows, ncols, i+1)
    idx = np.random.randint(low=0, high=X.shape[0])
    plt.plot(data[idx])
    text = "normal" if y[idx] == 1 else "abnormal"
    plt.title(f"label (y) = {int(y[idx])}: {text}")
    plt.xticks(np.linspace(0, 120, 5))
    plt.grid(True)

plt.tight_layout()
```



Next, we split the original dataset into two sets: training set and test set. We can use any splitting proportion we want. Here, we split $80\% - 20\%$ as above.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42)
n_samples_train, n_samples_test, n_samples = X_train.shape[0], X_test.shape[0], X.shape[0]
ratio = (n_samples_train/n_samples, n_samples_test/n_samples)

print(f"Number of samples in training set: {n_samples_train}")
print(f"Number of samples in test set: {n_samples_test}")
print(f"Split ratio = {ratio[0]:.4f}, {ratio[1]:.4f}")
```

```
Number of samples in training set: 3997
Number of samples in test set: 1000
Split ratio = 0.7999, 0.2001
```

Now, we define a `LogisticRegression` model object and perform the training

```
from sklearn.linear_model import LogisticRegression
log_reg = LogisticRegression()
log_reg.fit(X_train, y_train)
```

penalty	'l2'
dual	False
tol	0.0001
C	1.0
fit_intercept	True
intercept_scaling	1
class_weight	None
random_state	None
solver	'lbfgs'
max_iter	100

multi_class	'deprecated'
verbose	0
warm_start	False
n_jobs	None
l1_ratio	None

Unlike regression problems in which we can measure how well the model is by using R^2 score, we cannot use this score to measure how well the model performs on classification problem. Instead, we use the `accuracy_score` to measure how accurate the model can classify the samples into the right category. In our case, we measure how well the model can predict the samples in the test set. The score is just simply the number of correct predictions divided by the total number of predictions made.

```
from sklearn.metrics import accuracy_score
y_pred = log_reg.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Model accuracy: {accuracy: .4f}")
```

```
Model accuracy:  0.9870
```

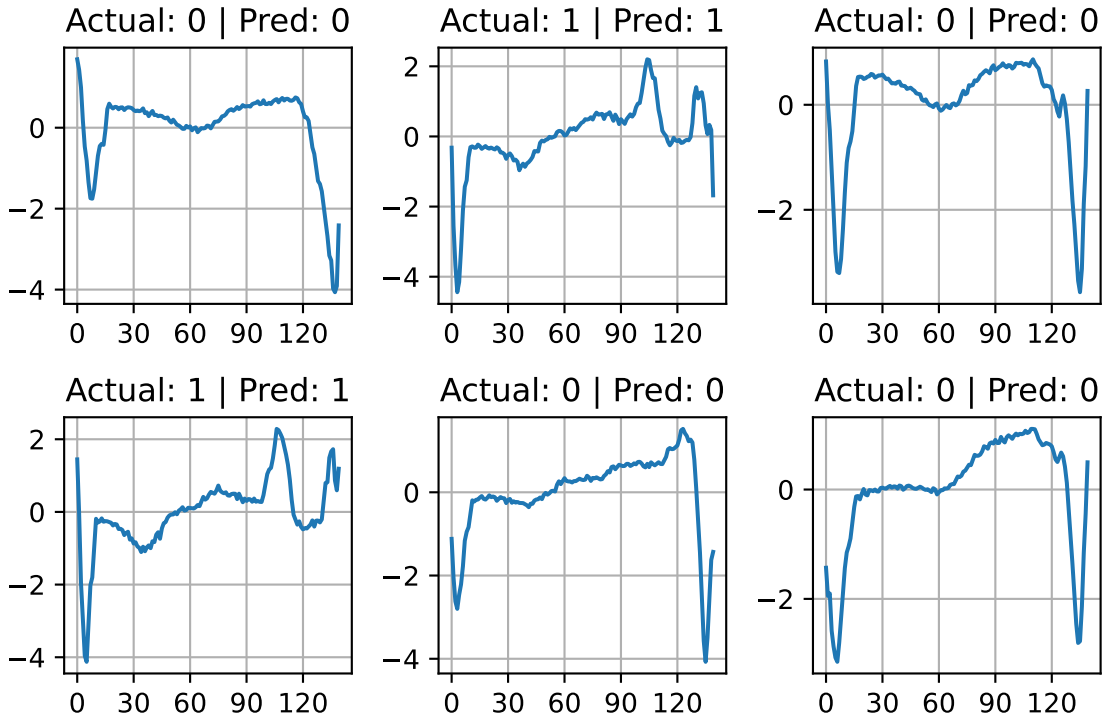
The model can make prediction fantastically, almost 99% correctly. Now, we plot some data in the test set and show both the prediction as well as the ground truth label (the labels that are given by the data).

```
nrows, ncols = 2, 3
plt.figure(figsize=(6, 4))
np.random.seed(seed=42)         # just for reproducibility purpose
for i in range(nrows * ncols):
    plt.subplot(nrows, ncols, i+1)

    idx = np.random.randint(low=0, high=X_test.shape[0])
    actual_label = int(y_test[idx])
    # convert this to a row vector which is a (1 x n) matrix.
    data_point = X_test[idx].reshape(1, -1)
    predicted_label = int(log_reg.predict(data_point)[0])

    plt.plot(X_test[idx])
    plt.title(f"Actual: {actual_label} | Pred: {predicted_label}")
    plt.xticks(np.linspace(0, 120, 5))
    plt.grid(True)

plt.tight_layout()
```



2.2 Hand-written digit classification

In this application, given an image of hand-written digit, we predict which number the image represents. This is a 10-category classification problem because we classify the image into 10 category: digit 0, digit 1, ..., digit 9.

Generate `.csv` data files

Fetch data online

First, we fetch the data online. This dataset has been used for benchmarking many machine learning models. The original dataset has 70000 gray-scale image, all images have the same size 28×28. If we "unroll" the matrix of size 28×28 into a single vector, this vector has size $28 \cdot 28 = 784$. So, one single sample has 784 features.

```
import pandas as pd
from sklearn.datasets import fetch_openml

print("Fetching full MNIST dataset (this step takes some time)...")
mnist = fetch_openml('mnist_784', version=1, as_frame=True, parser='auto') # mnist is a Bunch object
df = mnist.frame    # attribute frame is a dataframe
print(f"type(mnist) = {type(mnist)}")
print(f"type(df) = {type(df)}")
```

```
Fetching full MNIST dataset (this step takes some time)...
type(mnist) = <class 'sklearn.utils._bunch.Bunch'>
type(df) = <class 'pandas.DataFrame'>
```

Scale pixel values to double in $[0, 1]$

To make computing easy with `double` values in C++, I decide to scale the data into the `float64` (in Python) in the range $[0, 1]$.

```
pixel_columns = df.columns.drop('class')
df[pixel_columns] = df[pixel_columns].astype('float64') / 255.0
```

Write `DataFrame` objects into `.csv` files

To make the project easier, I write the fetched data into csv files so that you can read the `csv` file for data. In addition, I have created three datasets and provided the Python code here too:

- `./data/mnist_full.csv`: Original dataset of 70000 data points (images),
- `./data/mnist_mini.csv`: Subset of the original dataset with 10000 data points (images) – 1000 images for each digit
- `./data/mnist_mini.csv`: Subset of the original dataset with 3000 data points (images) – 300 images for each digit

> **Important**
>
> The subset can be used to benchmark your implementation. Instead of training the entire logistic regression model on a full dataset of 70000 images, you can test your code on its "mini subset" `mnist_mini.csv` with 10000 images or its "micro subset" `mnist_micro.csv` with 3000 images. This will significantly save you time in debugging your code.

First we create a `./data/mnist_full.csv` file using all 70000 images in the original dataset.

```
from pathlib import Path
file_path = Path("./data/mnist_full.csv")
if file_path.exists():
    print(f"File '{file_path}' already exists. Skipping saving full dataset.")
else:
    print("Saving full dataset to 'mnist_full.csv'...")
    df.to_csv(file_path, index=False, header=False)
```

```
File 'data/mnist_full.csv' already exists. Skipping saving full dataset.
```

Next, we create two files `./data/mnist_mini.csv` (10000 images) and `./data/mnist_micro.csv` (3000 images).

```
def generate_mnist_csv(df, n_per_digit, output_filename):

    file_path = Path(output_filename)
    if file_path.exists():
        print(f"File '{file_path}' already exists. Skipping generation of subset.")
        return

    # If the file does not exist, we proceed to generate the subset
    subsets = []
    print(f"Sampling {n_per_digit} images per digit for '{output_filename}'...")
    for digit in range(10):
        # Filter for the specific digit
        digit_df = df[df['class'].astype(int) == digit]
        # Randomly sample the required number
        sample = digit_df.sample(n=n_per_digit, random_state=42)
        subsets.append(sample)

    # Combine and save (keeping them grouped initially for structure)
    final_df = pd.concat(subsets).reset_index(drop=True)
    final_df.to_csv(output_filename, index=False, header=False)
    print(f"File '{output_filename}' saved with shape: {final_df.shape}")

n_per_digit = [1000, 300]  # number of images per digit for the two subsets
```

```
output_filenames = ['./data/mnist_mini.csv', './data/mnist_micro.csv']
for n, output_filename in zip(n_per_digit, output_filenames):
    generate_mnist_csv(df, n, output_filename)
```

```
File 'data/mnist_mini.csv' already exists. Skipping generation of subset.
File 'data/mnist_micro.csv' already exists. Skipping generation of subset.
```

All the steps described above are given in one single python file `./src/generate_mnist_csv.py`.

Visualize data

Let us read the `.csv` file and visualize the data by showing a few samples in images. To do this, we need to reshape a 1-dimensional vector of size 784 into a 2-dimensional array of size 28×28. Then, we shall use `.imshow()` member function of the `axis` object to show the images.

First, we read the `.csv` file into a `DataFrame` object, and then convert the data in this dataframe into `ndarray` objects.

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv('./data/mnist_micro.csv', header=None)

# Shuffle the data so that we can visualize various digits
df_shuffled = df.sample(frac=1, random_state=42).reset_index(drop=True)

# Split features and labels -- in numpy arrays
X = df_shuffled.iloc[:,:-1].values
y = df_shuffled.iloc[:, -1].values

print(f"type(df) = {type(df)}")
print(f"type(X) = {type(X)}")
print(f"type(y) = {type(y)}")

print(f"X.shape = {X.shape}")
print(f"y.shape = {y.shape}")
```

```
type(df) = <class 'pandas.DataFrame'>
type(X) = <class 'numpy.ndarray'>
type(y) = <class 'numpy.ndarray'>
X.shape = (3000, 784)
y.shape = (3000,)
```

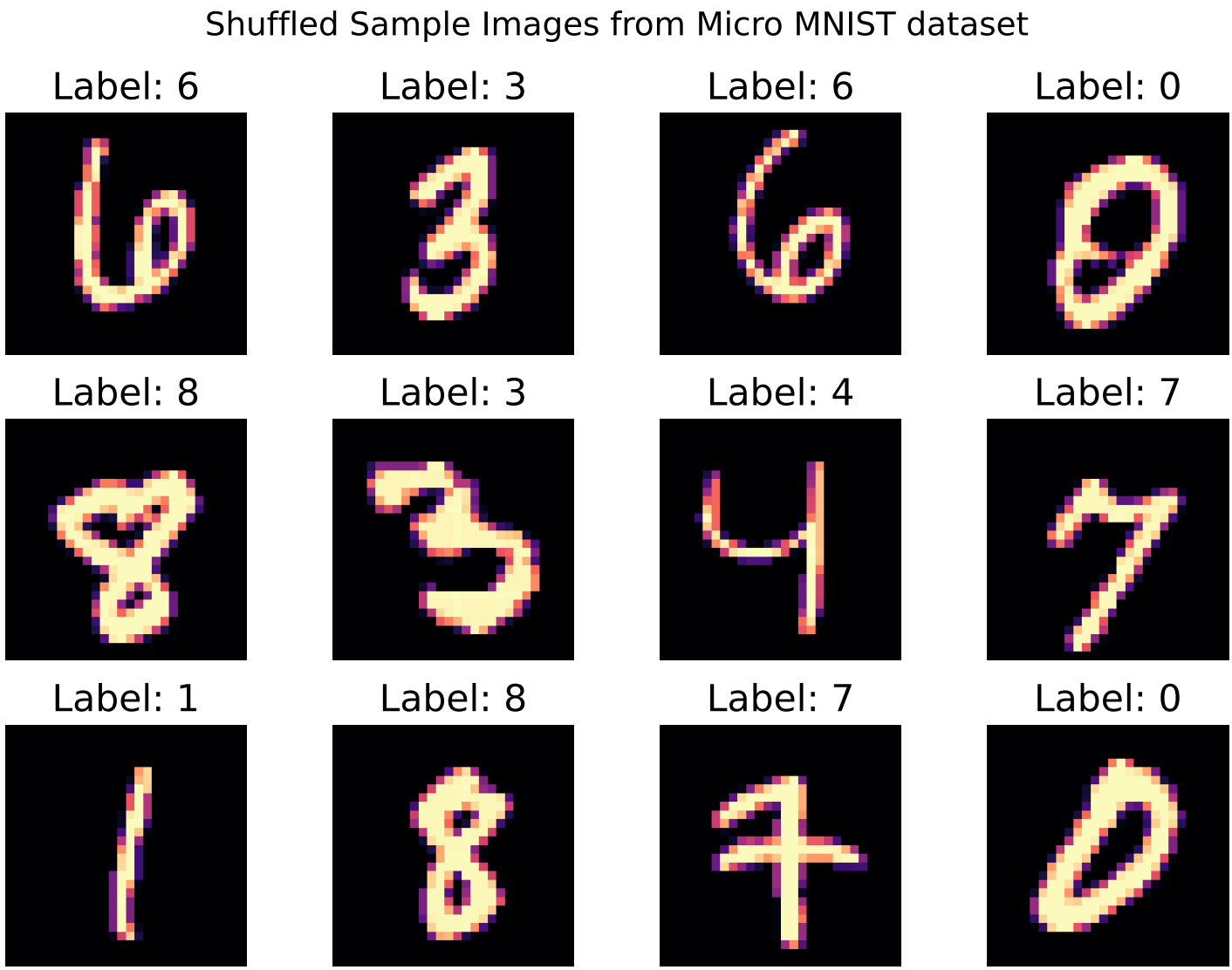
Now that we show 12 images in the a grid of 3×4 subplots.

```
fig, axes = plt.subplots(3, 4, figsize=(8, 6))
fig.suptitle("Shuffled Sample Images from Micro MNIST dataset", fontsize=14)
for i, ax in enumerate(axes.flatten()):
    image = X[i].reshape(28, 28)

    # Display the image with magma colormap.
    # A few colormap to try out:
    #   viridis (default), plasma, inferno, magma, cividis, gray
    ax.imshow(image, 'magma')
    # avoid floating point presentation by casting to int: int(y[i])
```

```
    ax.set_title(f"Label: {int(y[i])}", fontsize=16)
    ax.axis('off')

plt.tight_layout()
plt.show()
```



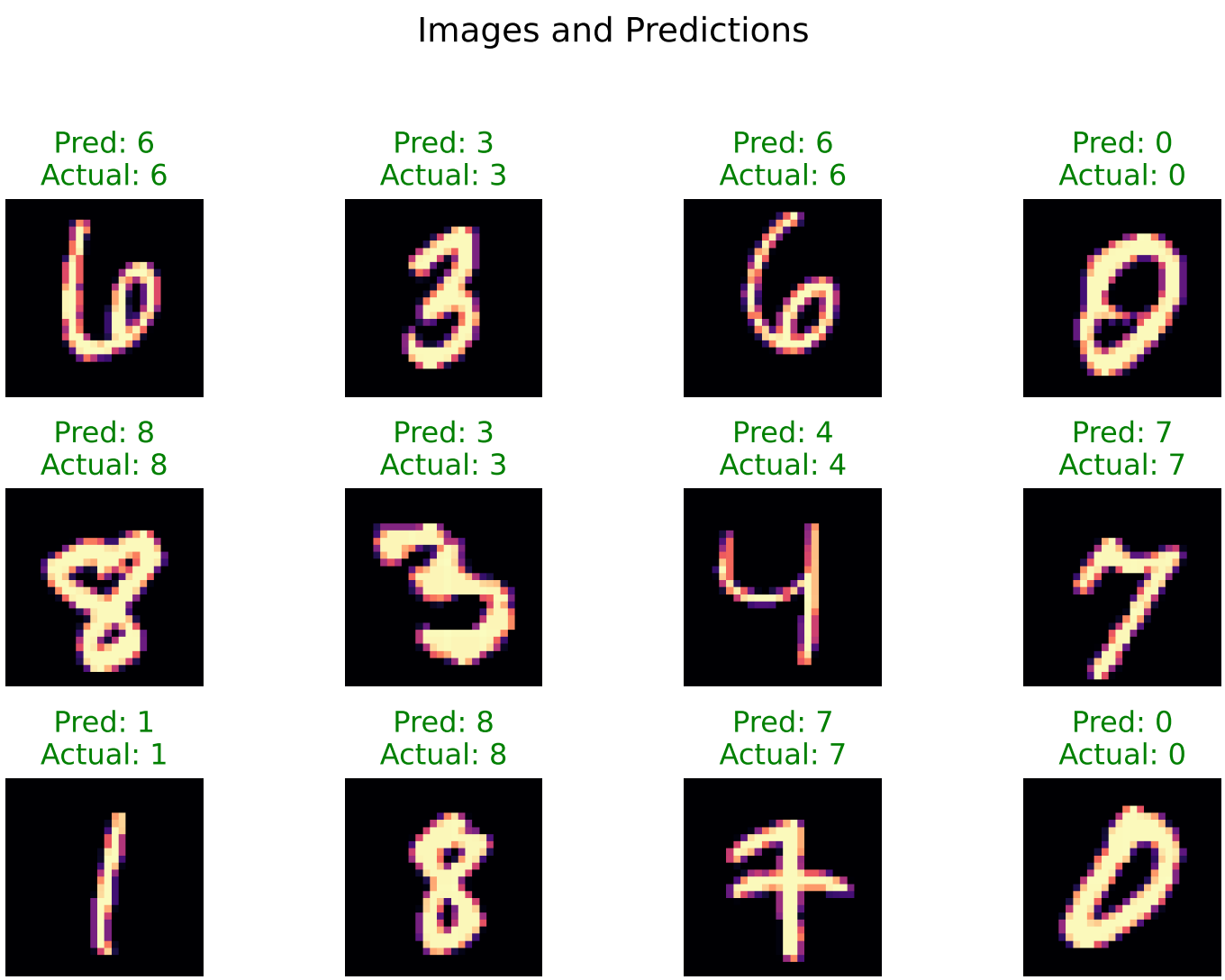
The code snippets mentioned in this step are given in the file `show_images.py`.

Train the model

Training the Logistic Regressio from **scikit-learn**, namely `LogisticRegression` object, is easy.

```
# For this example, We don't need to split the dataset into training
# set and test set. We just use all the given data here. We use
# max_iter=1000 to ensure convergence on the given dataset
model = LogisticRegression(max_iter=1000)
model.fit(X, y)
# Make prediction on the entire set of images
y_predict = model.predict(X)
```

Visualization of images and prediction

We visualize some images in the dataset and show the actual label as well as he predicted label. The code is very similar to the code snippet above for visualization of images in the dataset. The key difference is we need to compute the label predictions and show them in the figure.

```
fig, axes = plt.subplots(3, 4, figsize=(8, 6))
fig.suptitle("Images and Predictions", fontsize=14)

accuracy = accuracy_score(y_true=y, y_pred=y_predict)
for i, ax in enumerate(axes.flatten()):
    image = X[i].reshape(28, 28)
    ax.imshow(image, cmap="magma")

    actual, predicted = y[i], y_predict[i]
    color = 'green' if predicted == actual else 'red'
    ax.set_title(f"Pred: {int(predicted)}\nActual: {int(actual)}", color=color)

    ax.axis(False)
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()
```